

PROMETEO



GoFight: Aplicación móvil para la gestión y aprendizaje
de clases de boxeo.

Técnico Superior en Desarrollo de Aplicaciones
Multiplataforma (DAM) / Online

Alumnos:

Mario Hernández Moreno

Ayoub Arramdani

Agustín Linares Carrera

Tutor: Luz María Álvarez Moreno

DEDICATORIA

A nuestras madres, María del Rosario Moreno Rocamora, Emilia Carrera Fernández y Saida Arramdani Hamoumi, por todo lo que han hecho por nosotros. Por su esfuerzo constante, por estar siempre ahí incluso en los momentos más difíciles y por enseñarnos a no rendirnos nunca. Gracias por empujarnos a superarnos cada día y por confiar en nosotros incluso cuando nosotros mismos dudábamos.

Y a nuestra profesora Luz María Álvarez Moreno, por acompañarnos durante todo este camino. Gracias por su paciencia, por su cercanía y por cada consejo que nos ha dado, que ha sido clave para poder sacar este trabajo adelante de la mejor manera posible.

ÍNDICE

ÍNDICE	3
RESUMEN	4
ABSTRACT	4
JUSTIFICACIÓN DEL PROYECTO	6
INTRODUCCIÓN	7
OBJETIVOS	9
R01 – El sistema debe permitir la gestión de usuarios	9
R01F01 – Registro de usuarios	9
R01F02 – Inicio de sesión	9
R02 – El sistema debe permitir la gestión de rutinas de entrenamiento	10
R02F01 – Visualización de rutinas	10
R02F02 – Visualización de ejercicios	10
R03 – El sistema debe registrar sesiones de entrenamiento	10
R03F01 – Registro de sesiones	10
R04 – El sistema debe incluir un sistema de gamificación	11
R04F01 – Sistema de puntos y rachas	11
DISEÑOS	12
Arquitectura de la solución	12
Casos de uso	13
Interfaces	24
TECNOLOGÍA	44
METODOLOGÍA	46
Diagrama de Gantt	48
README y GIT	49
TRABAJOS FUTUROS	50
CONCLUSIONES	52
REFERENCIAS	54

RESUMEN

Este Trabajo de Fin de Grado presenta el desarrollo de **GoFight**, una aplicación móvil orientada a la gestión y seguimiento de entrenamientos de boxeo. El objetivo principal del proyecto es ofrecer una herramienta que permita a los usuarios organizar sus rutinas de entrenamiento, registrar sesiones realizadas y consultar su progreso de forma sencilla.

La aplicación permite a los usuarios registrarse e iniciar sesión, acceder a diferentes rutinas de entrenamiento y visualizar los ejercicios que componen cada rutina. Además, se ha incorporado un sistema de **gamificación** basado en rachas de entrenamiento y acumulación de puntos, con el objetivo de fomentar la constancia y la motivación de los usuarios.

A nivel técnico, el sistema se ha desarrollado utilizando **Node.js** para el backend, **PostgreSQL** como sistema de gestión de bases de datos y **Prisma ORM** para la gestión del modelo de datos y la comunicación con la base de datos.

ABSTRACT

This Final Degree Project presents the development of **GoFight**, a mobile application focused on the management and tracking of boxing training sessions. The main objective of the project is to provide a tool that allows users to organize their training routines, register completed sessions and monitor their progress in a simple way.

The application allows users to register and log in, access different training routines and view the exercises included in each routine. In addition, a **gamification system** based on training streaks and points has been implemented in order to encourage user motivation and consistency.

From a technical perspective, the system has been developed using **Node.js** for the backend, **PostgreSQL** as the database management system and **Prisma ORM** to handle the data model and database communication.

PROMETEO

JUSTIFICACIÓN DEL PROYECTO

En los últimos años, el uso de aplicaciones móviles relacionadas con el deporte y el entrenamiento personal ha aumentado considerablemente. Estas herramientas permiten a los usuarios organizar sus rutinas, registrar su actividad física y realizar un seguimiento de su progreso de forma sencilla y accesible desde cualquier lugar. Sin embargo, muchas de estas aplicaciones están enfocadas al entrenamiento general y no ofrecen funcionalidades específicas orientadas a deportes concretos como el boxeo.

El proyecto **GoFight** surge con el objetivo de desarrollar una aplicación que permita gestionar y organizar entrenamientos de boxeo de manera sencilla e intuitiva. A través de esta aplicación, los usuarios pueden registrarse en el sistema, acceder a diferentes rutinas de entrenamiento y consultar los ejercicios que componen cada rutina. Además, el sistema permite registrar sesiones realizadas, lo que facilita el seguimiento del progreso de cada usuario.

Otra de las características destacadas de la aplicación es la incorporación de un sistema de **gamificación**, que incluye rachas de entrenamiento y un sistema de puntos. Este enfoque busca motivar a los usuarios a mantener la constancia en sus entrenamientos, fomentando hábitos deportivos saludables mediante mecánicas similares a las utilizadas en otras aplicaciones de productividad o actividad física.

Actualmente existen aplicaciones de entrenamiento físico como **Nike Training Club, Fitbod o Freeletics**, que ofrecen rutinas de ejercicio y seguimiento de actividad. No obstante, muchas de ellas no están específicamente orientadas al boxeo ni permiten una personalización sencilla de rutinas centradas en este deporte. Por este motivo, GoFight pretende ofrecer una alternativa centrada en el entrenamiento de boxeo, combinando gestión de rutinas, seguimiento del progreso y elementos de motivación mediante gamificación.

De esta manera, el proyecto no solo permite aplicar los conocimientos adquiridos durante el ciclo formativo de **Desarrollo de Aplicaciones Multiplataforma (DAM)**, sino que también propone una solución práctica orientada al ámbito deportivo que puede ser ampliada con nuevas funcionalidades en el futuro.

INTRODUCCIÓN

El desarrollo de aplicaciones móviles orientadas al deporte y al entrenamiento personal ha experimentado un crecimiento significativo en los últimos años. Estas aplicaciones permiten a los usuarios planificar sus entrenamientos, registrar su actividad física y realizar un seguimiento de su progreso de forma cómoda desde sus dispositivos móviles. En este contexto surge **GoFight**, una aplicación diseñada específicamente para facilitar la organización y seguimiento de entrenamientos de boxeo.

El objetivo principal de GoFight es proporcionar una herramienta digital que permita a los usuarios gestionar sus rutinas de entrenamiento de forma sencilla e intuitiva. A través de la aplicación, los usuarios pueden registrarse en el sistema, iniciar sesión y acceder a diferentes rutinas de entrenamiento compuestas por ejercicios específicos relacionados con el boxeo. Además, la aplicación permite registrar las sesiones realizadas, lo que facilita el seguimiento de la actividad y el progreso del usuario.

Una de las características principales del proyecto es la incorporación de un sistema de **gamificación**, que incluye elementos como rachas de entrenamiento y acumulación de puntos. Estos mecanismos tienen como finalidad motivar a los usuarios a mantener la constancia en sus entrenamientos, incentivando la participación y el compromiso con la actividad física.

Desde el punto de vista técnico, el proyecto se basa en una arquitectura cliente-servidor. El backend ha sido desarrollado utilizando **Node.js**, mientras que la gestión de la base de datos se realiza mediante **PostgreSQL**, utilizando **Prisma ORM** para facilitar la comunicación entre la aplicación y el sistema de almacenamiento de datos. Esta combinación de tecnologías permite desarrollar una aplicación escalable, mantenible y preparada para futuras ampliaciones.

Entre las principales funcionalidades que ofrece la aplicación se encuentran:

- Registro e inicio de sesión de usuarios.

- Gestión de rutinas de entrenamiento.
- Consulta de ejercicios asociados a cada rutina.
- Registro de sesiones de entrenamiento realizadas.
- Sistema de gamificación basado en rachas y puntos.
- Visualización del progreso del usuario.

De esta forma, GoFight representa una solución tecnológica que combina gestión de entrenamientos, seguimiento del progreso y motivación del usuario dentro de una misma plataforma, aplicando los conocimientos adquiridos durante el ciclo formativo de **Desarrollo de Aplicaciones Multiplataforma (DAM)**.

OBJETIVOS

R01 – El sistema debe permitir la gestión de usuarios

R01F01 – Registro de usuarios

El sistema debe permitir que nuevos usuarios se registren introduciendo sus datos personales.

R01F01T01 – Crear la estructura de la tabla de usuarios en la base de datos PostgreSQL.

R01F01T01P01 – Verificar que la tabla de usuarios se ha creado correctamente con los campos definidos.

R01F01T02 – Implementar el formulario de registro en la aplicación.

R01F01T02P01 – Comprobar que el formulario muestra correctamente todos los campos necesarios.

R01F01T03 – Validar los datos introducidos por el usuario antes de almacenarlos.

R01F01T03P01 – Verificar que el sistema impide el registro con datos incompletos o incorrectos.

R01F02 – Inicio de sesión

El sistema debe permitir que los usuarios registrados puedan iniciar sesión en la aplicación.

R01F02T01 – Crear el formulario de inicio de sesión.

R01F02T01P01 – Comprobar que el formulario de acceso se visualiza correctamente.

R01F02T02 – Comprobar que las credenciales coinciden con los datos almacenados en la base de datos.

R01F02T02P01 – Verificar que solo se permite el acceso cuando las credenciales son correctas.

R02 – El sistema debe permitir la gestión de rutinas de entrenamiento

R02F01 – Visualización de rutinas

El sistema debe permitir que los usuarios puedan consultar las rutinas disponibles.

R02F01T01 – Crear la estructura de la tabla de rutinas en la base de datos.

R02F01T01P01 – Verificar que la tabla de rutinas se ha creado correctamente.

R02F01T02 – Mostrar las rutinas disponibles en la interfaz de la aplicación.

R02F01T02P01 – Comprobar que las rutinas se muestran correctamente en pantalla.

R02F02 – Visualización de ejercicios

El sistema debe permitir consultar los ejercicios asociados a cada rutina.

R02F02T01 – Crear la tabla de ejercicios en la base de datos.

R02F02T01P01 – Verificar que la tabla de ejercicios se ha creado correctamente.

R02F02T02 – Relacionar los ejercicios con las rutinas mediante una tabla intermedia.

R02F02T02P01 – Comprobar que cada rutina muestra únicamente los ejercicios que tiene asociados.

R03 – El sistema debe registrar sesiones de entrenamiento

R03F01 – Registro de sesiones

El sistema debe permitir almacenar el historial de entrenamientos realizados por los usuarios.

R03F01T01 – Crear la tabla de historial de sesiones.

R03F01T01P01 – Verificar que la tabla de historial se ha creado correctamente.

R03F01T02 – Registrar la fecha de entrenamiento y los datos asociados.

R03F01T02P01 – Comprobar que las sesiones realizadas quedan almacenadas correctamente con su fecha correspondiente.

R04 – El sistema debe incluir un sistema de gamificación

R04F01 – Sistema de puntos y rachas

El sistema debe permitir registrar puntos y rachas de entrenamiento para incentivar la constancia de los usuarios.

R04F01T01 – Crear la tabla de gamificación en la base de datos.

R04F01T01P01 – Verificar que la tabla de gamificación se ha creado correctamente.

R04F01T02 – Implementar el cálculo de rachas de entrenamiento y puntos acumulados.

R04F01T02P01 – Comprobar que los puntos y las rachas se actualizan correctamente tras cada sesión registrada.

...

DISEÑOS

Arquitectura de la solución.

La arquitectura de la aplicación **GoFight** sigue un modelo cliente-servidor, en el cual la aplicación móvil actúa como cliente que se comunica con un servidor encargado de gestionar la lógica del sistema y el acceso a la base de datos.

El cliente permite a los usuarios interactuar con la aplicación mediante una interfaz gráfica que facilita el acceso a las diferentes funcionalidades del sistema, como el registro de usuarios, la consulta de rutinas de entrenamiento o el seguimiento del progreso.

El servidor se encarga de procesar las peticiones realizadas por los usuarios, gestionar la lógica de negocio de la aplicación y comunicarse con la base de datos para almacenar o recuperar información. Para ello se utiliza **Node.js** como entorno de ejecución del backend.

La información generada por el sistema se almacena en una base de datos relacional **PostgreSQL**, que permite gestionar de forma eficiente los datos relacionados con usuarios, rutinas de entrenamiento, ejercicios y sesiones registradas.

Para facilitar la interacción entre la aplicación y la base de datos se utiliza **Prisma ORM**, que permite definir el modelo de datos de forma estructurada y realizar consultas de forma segura y eficiente.

Esta arquitectura permite separar claramente las responsabilidades de cada componente del sistema, facilitando el mantenimiento, la escalabilidad y la ampliación futura de la aplicación:

Explicación del flujo: Puedes añadir que el uso de Prisma actúa como un puente que traduce los objetos de JavaScript a sentencias SQL complejas, optimizando el rendimiento de la aplicación.

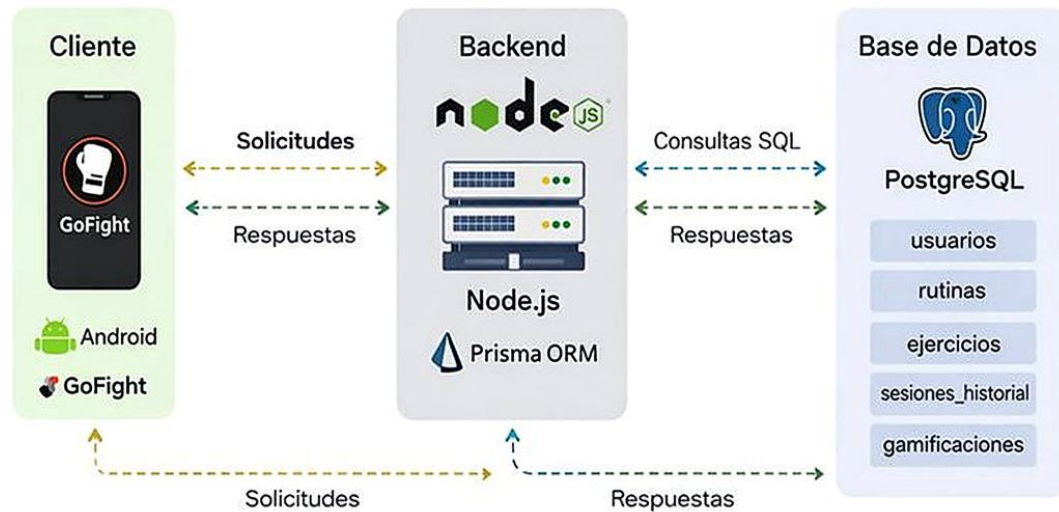


Imagen 1: Representación gráfica de la arquitectura desarrollada en relación al cliente, backend y la base de datos.

Casos de uso.

Para los casos de uso, es recomendable agruparlos por módulos (Gestión de Usuarios, Entrenamiento y Gamificación) para que el lector entienda la estructura lógica:

- **Módulo de Identidad:** Incluye el registro e inicio de sesión. Aquí es vital mencionar que la seguridad se gestiona mediante **JSON Web Tokens (JWT)**, lo que permite una comunicación sin estado (stateless) y segura.
- **Módulo de Entrenamiento:** Explica que el sistema no solo muestra datos, sino que filtra las rutinas según el perfil del usuario autenticado.
- **Módulo de Gamificación:** (Aunque aparece como postcondición en el registro) puedes redactar que: "Tras un registro exitoso, el sistema instancia automáticamente un registro en la tabla de gamificaciones, vinculando al usuario con un contador de rachas y puntos inicializado en cero para fomentar el *engagement* desde el primer momento".

Caso de uso: Registro del usuario

DESCRIPCIÓN: Permitir que un nuevo usuario cree una cuenta en la aplicación GoFight introduciendo sus datos personales.	
PRECONDICIONES: • El usuario no debe estar registrado previamente. • El servidor debe estar operativo. • Debe existir conexión con la base de datos.	POSTCONDICIONES: • El usuario queda registrado en el sistema. • Se crea su registro inicial de gamificación. • El usuario puede iniciar sesión posteriormente.
DATOS ENTRADA • Nombre • Email • Contraseña	DATOS SALIDA • Mensaje de registro correcto • Usuario creado • Token de autenticación
TABLAS: • usuarios • gamificaciones	CLASES: • PrismaClient • AuthController • jwt • uploads
INTERFACES: • Register • TextInput • Button • ErrorMsg	

Caso de uso: Inicio de sesión

DESCRIPCIÓN: Permitir que un usuario registrado acceda a la aplicación mediante su correo electrónico y contraseña.	
PRECONDICIONES: • El usuario debe existir en la base de datos. • El backend debe estar disponible. • La base de datos debe estar conectada.	POSTCONDICIONES: • El usuario obtiene acceso al sistema. • Se genera un token de autenticación. • Se redirige a la pantalla principal.
DATOS ENTRADA • Email • Contraseña	DATOS SALIDA • Token de autenticación • Mensaje de acceso correcto
TABLAS: • usuarios	CLASES: • PrismaClient • AuthController • Jwt
INTERFACES: • Login • TextInput • Button • ErrorMsg • Home	

Caso de uso: Ver perfil

DESCRIPCIÓN: Permitir que un usuario autenticado consulte su información personal dentro de la aplicación GoFight, mostrando los datos asociados a su cuenta.	
PRECONDICIONES: - El usuario debe haber iniciado sesión previamente. - Debe existir un token de autenticación válido almacenado en el sistema. - El servidor y la base de datos deben estar operativos.	POSTCONDICIONES: - El sistema muestra los datos del perfil del usuario autenticado. - El usuario puede consultar su información personal dentro de la aplicación.
DATOS ENTRADA Token de autenticación del usuario.	DATOS SALIDA - Identificador del usuario. - Nombre del usuario. - Correo electrónico. - Perfil del usuario. - Rol del usuario.
TABLAS: usuarios	CLASES: - PrismaClient - AuthController - AuthMiddleware - getUserProfile
INTERFACES: - Perfil - Home - HeaderComponent	

Caso de uso: Ver rutinas

DESCRIPCIÓN: Permitir que un usuario autenticado consulte las rutinas de entrenamiento disponibles asociadas a su cuenta.	
PRECONDICIONES: - El usuario debe haber iniciado sesión previamente y tener un token de autenticación válido. - El usuario debe estar registrado en la base de datos. - El servidor y la base de datos deben estar disponibles.	POSTCONDICIONES: - El sistema muestra el listado de rutinas disponibles del usuario. - El usuario puede consultar la información general de sus rutinas de entrenamiento.
DATOS ENTRADA Token de autenticación del usuario.	DATOS SALIDA - Identificador de la rutina. - Nombre de la rutina. - Dificultad de la rutina. - Identificador del usuario asociado.
TABLAS: - usuarios - rutinas	CLASES: - PrismaClient - RutinasController - AuthMiddleware - getRutinas
INTERFACES: - Rutinas - Home - RutinasRecomendadas - StackContainer	

Caso de uso: Ver ejercicios de una rutina

DESCRIPCIÓN: Permitir que un usuario autenticado consulte los ejercicios asociados a una rutina concreta, visualizando la información necesaria para realizar el entrenamiento.	
PRECONDICIONES: - El usuario debe haber iniciado sesión previamente y tener un token de autenticación válido. - La rutina consultada debe existir en la base de datos y debe tener ejercicios asociados. - El servidor y la base de datos deben estar operativos.	POSTCONDICIONES: - El sistema muestra el listado de ejercicios correspondientes a la rutina seleccionada. - El usuario puede consultar la información de cada ejercicio incluido en la rutina.
DATOS ENTRADA - Token de autenticación del usuario. - Identificador de la rutina seleccionada.	DATOS SALIDA - Identificador del ejercicio. - Nombre del ejercicio. - Categoría del ejercicio. - URL del vídeo explicativo. - Duración del ejercicio. - Duración del descanso. - Orden del ejercicio dentro de la rutina.
TABLAS: - ejercicios - rutinas - rutinas_ejercicios	CLASES: - PrismaClient - RutinasController - AuthMiddleware

	EjerciciosController
INTERFACES: - Rutinas - RutinasRecomendadas - StackContainer	

Diseño de Datos: Diagrama Entidad-Relación (E/R)

El modelo de datos de **GoFight** ha sido diseñado bajo una estructura relacional para garantizar la integridad referencial y la escalabilidad del sistema. El diagrama se centra en la gestión de usuarios, la planificación de entrenamientos de boxeo y el sistema de gamificación.

Descripción de las Entidades Principales

Para comprender la lógica del sistema, se describen a continuación las entidades clave representadas en el modelo:

- Usuario (User):** Es el núcleo del sistema. Almacena la información de identidad (nombre, correo electrónico y contraseña encriptada) y sirve como punto de anclaje para el progreso y la personalización. Cada usuario posee un perfil único que vincula sus logros y rutinas.
- Rutina (Routine):** Representa un conjunto organizado de actividades físicas. Las rutinas están categorizadas por niveles (principiante, intermedio, avanzado) para permitir una progresión lógica en el aprendizaje del boxeo.
- Ejercicio (Exercise):** Es la unidad mínima de entrenamiento. Contiene metadatos como el nombre, la descripción técnica y la categoría (fuerza, técnica, cardio).
- Gamificación (Gamification):** Entidad vinculada de forma unívoca (1:1) al usuario. Registra métricas de fidelización como "rachas" (días consecutivos de entrenamiento) y "puntos totales", permitiendo la implementación de mecánicas de juego en la aplicación.

Relaciones y Reglas de Negocio

La potencia del diseño reside en cómo estas entidades se conectan para resolver las necesidades del usuario:

1. **Relación Usuario - Rutina (N:M):** Un usuario puede estar inscrito o seguir múltiples rutinas, y una rutina puede ser realizada por muchos usuarios. Esta relación permite que el sistema ofrezca planes de entrenamiento predefinidos o personalizados.
2. **Composición de Rutinas (Tabla Intermedia Routine_Exercise):** Para dotar de dinamismo al sistema, se ha implementado una relación de muchos a muchos entre Rutinas y Ejercicios. Esta tabla intermedia no solo vincula ambas entidades, sino que añade atributos críticos para la ejecución:
 - **Orden:** Determina la secuencia lógica del entrenamiento (ej. primero calentamiento, luego técnica).
 - **Duración/Series:** Define la intensidad específica de ese ejercicio dentro de esa rutina concreta.
3. **Registro de Sesiones (User_Exercise_Log):** Cada vez que un usuario completa un ejercicio, se genera una entrada en esta tabla. Esto permite realizar un seguimiento histórico y es la fuente de datos principal para alimentar el sistema de gamificación.

Justificación del Diseño

El uso de un modelo relacional en **PostgreSQL** permite que el sistema maneje de forma eficiente la integridad de los datos. Por ejemplo, mediante el uso de **claves foráneas (FK)** con borrado restringido o en cascada, nos aseguramos de que no queden registros huérfanos (como ejercicios sin una rutina asociada), garantizando una experiencia de usuario sin errores críticos en la navegación.

PROMETEO

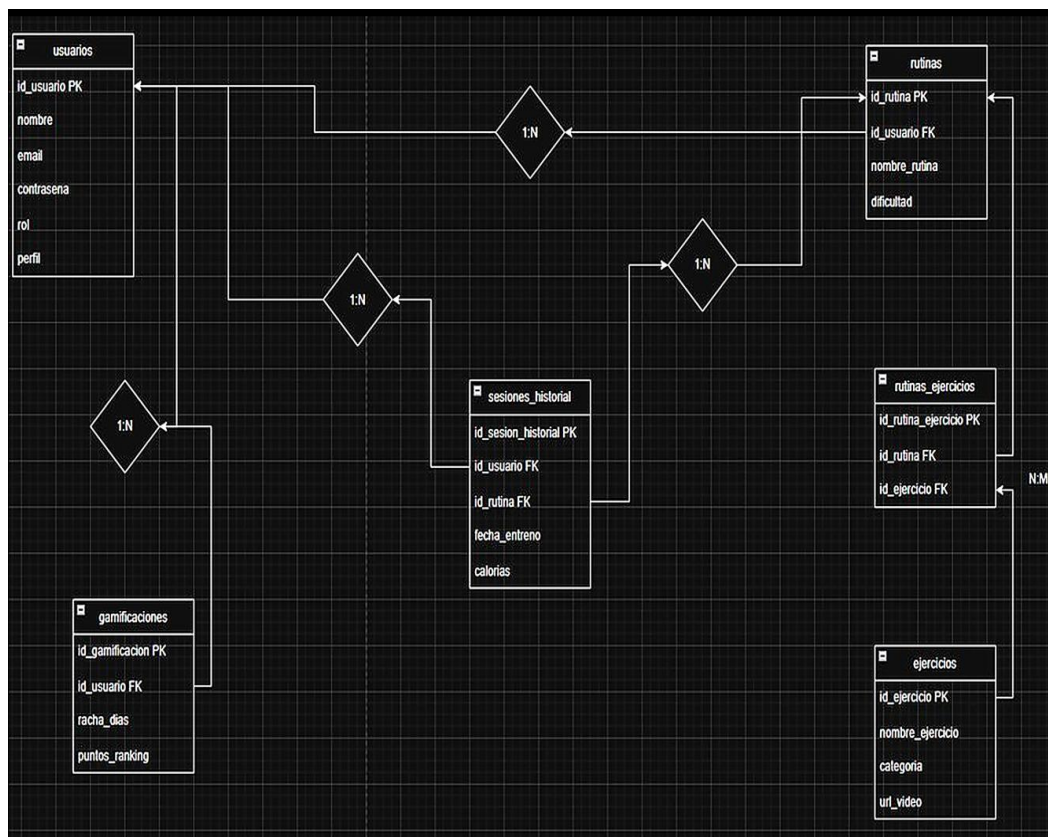


Imagen 2: Representación gráfica del diagrama E/R (Entidad Relación).

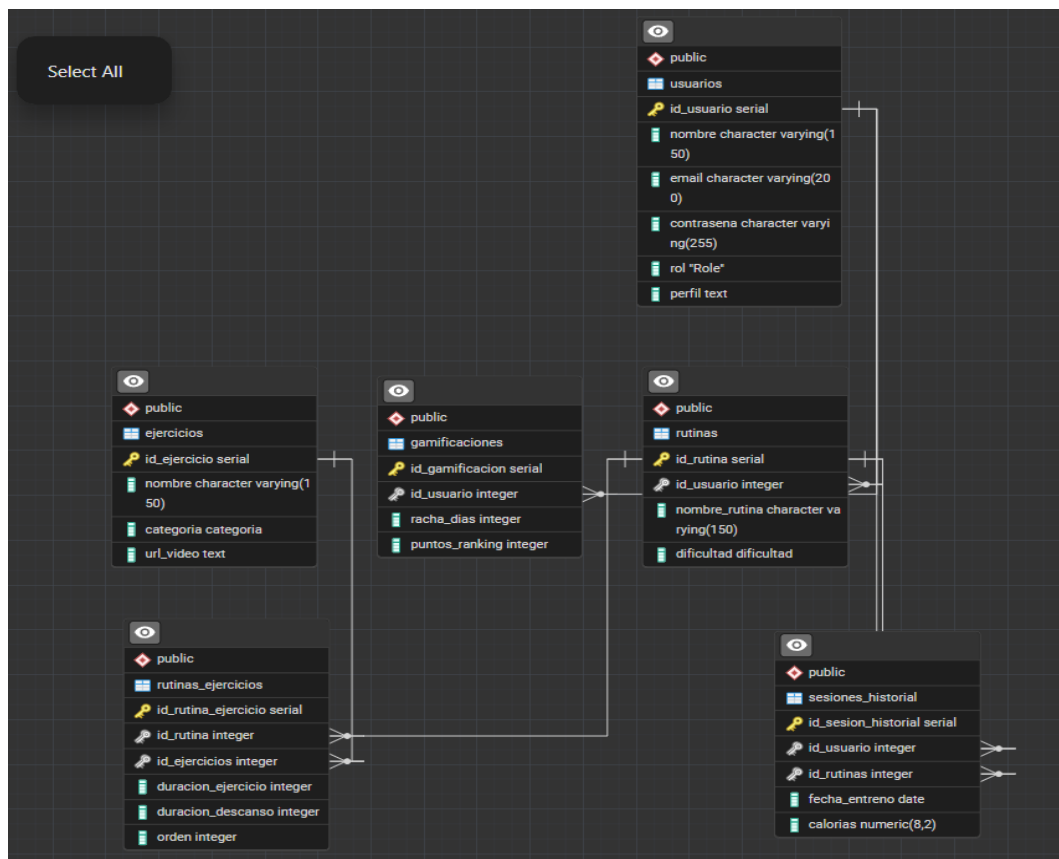


Imagen 3: Representación gráfica del diagrama de la base de datos y diagrama de clases con detalle de campos.

Lógica de Secuenciación y Tiempos en Entrenamientos

Una de las piezas clave en el diseño de **GoFight** es la capacidad de guiar al usuario de forma dinámica durante su sesión de boxeo. Esta funcionalidad se apoya en una estructura de datos avanzada en la relación entre **Rutinas** y **Ejercicios**:

- **Atributo orden:** Se ha incorporado un campo secuencial en la tabla intermedia que define la posición exacta de cada ejercicio dentro de una rutina. Esto permite al frontend renderizar el entrenamiento de forma cronológica (calentamiento, parte principal y vuelta a la calma).
- **Atributo duracion_ejercicio:** Define el tiempo de ejecución en segundos para cada bloque de trabajo. Este campo es consumido por la aplicación móvil para activar temporizadores automáticos, eliminando la necesidad de que el usuario manipule el dispositivo durante el ejercicio.
- **Atributo duracion_descanso:** Al igual que en el entrenamiento real de boxeo, se ha modelado el tiempo de recuperación entre series. Esta lógica permite automatizar los intervalos de descanso, fundamentales para el cumplimiento de los objetivos deportivos de la aplicación.

PROMETEO

Gracias a este diseño, el sistema no solo muestra información estática, sino que se convierte en un asistente de entrenamiento reactivo que gestiona los tiempos del deportista.

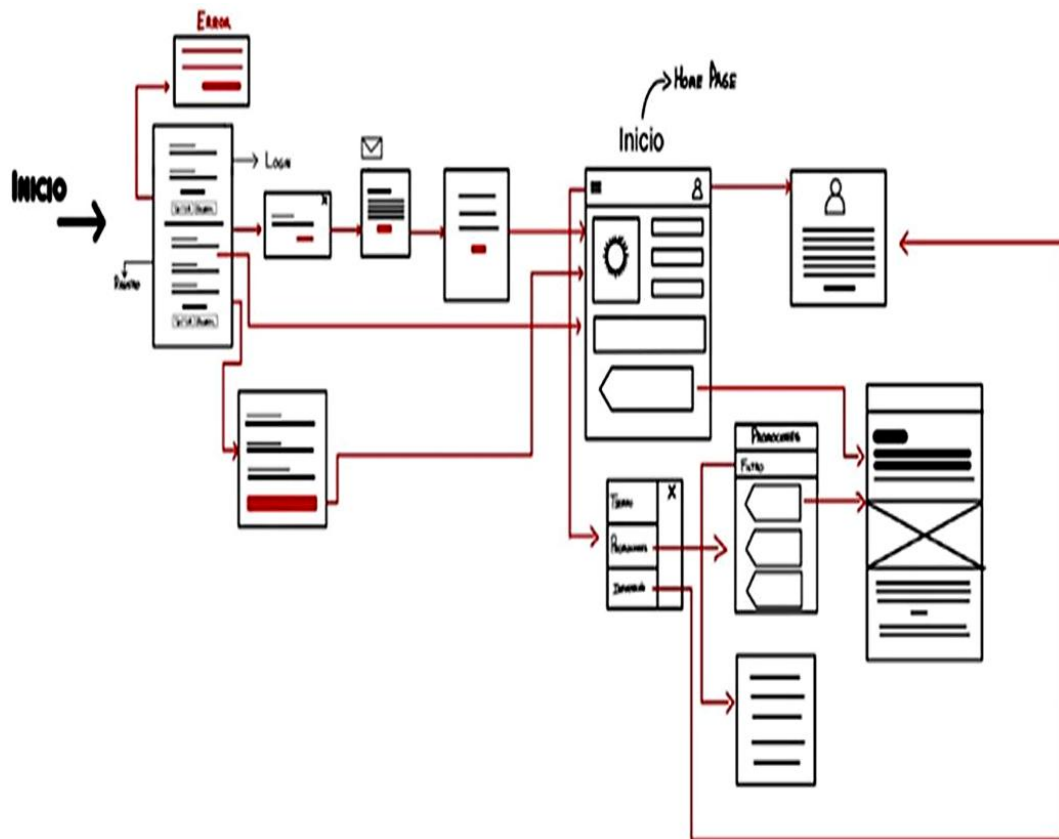


Imagen 4: Representación gráfica del diagrama del flujo de navegación.

Interfaces.

REGISTRO

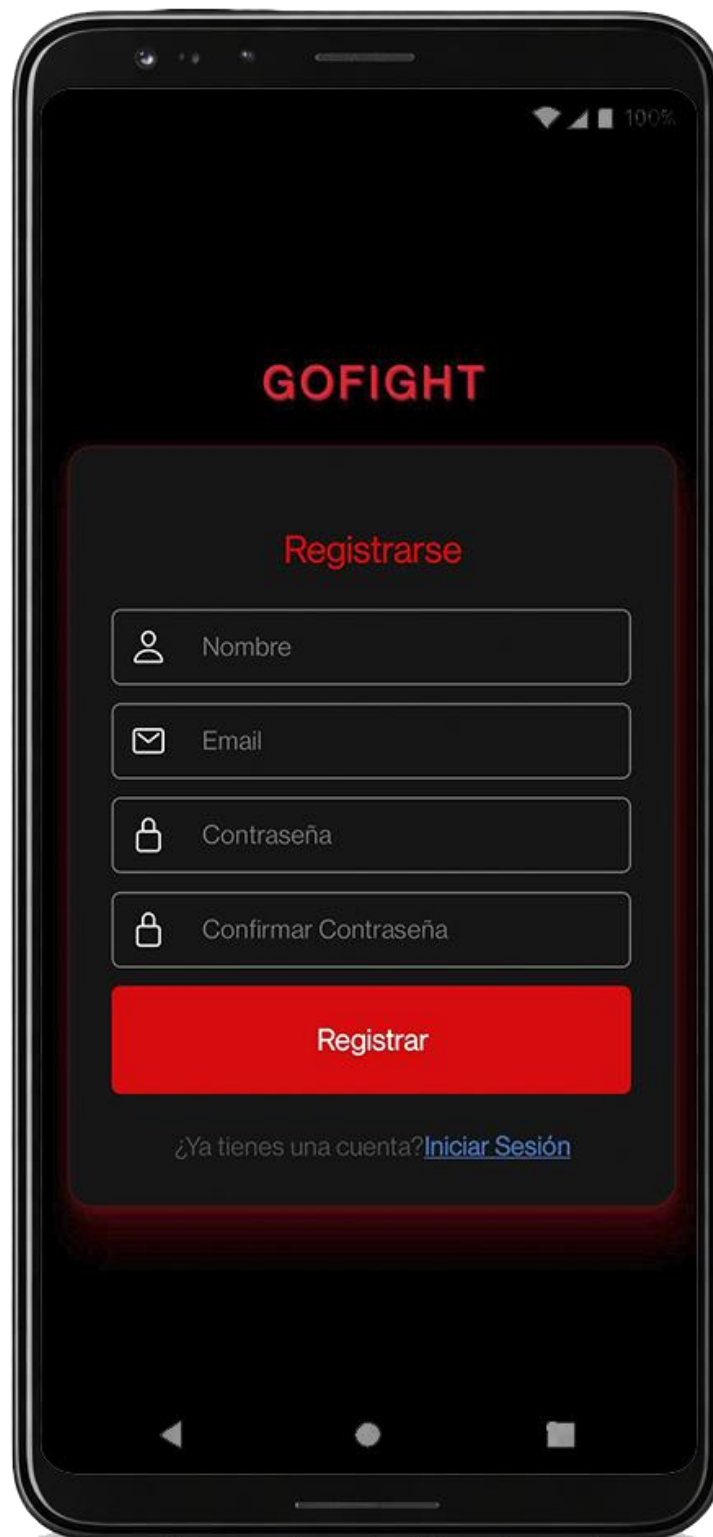


Imagen 5: Captura de pantalla del registro de un nuevo usuario.

INICIO DE SESIÓN

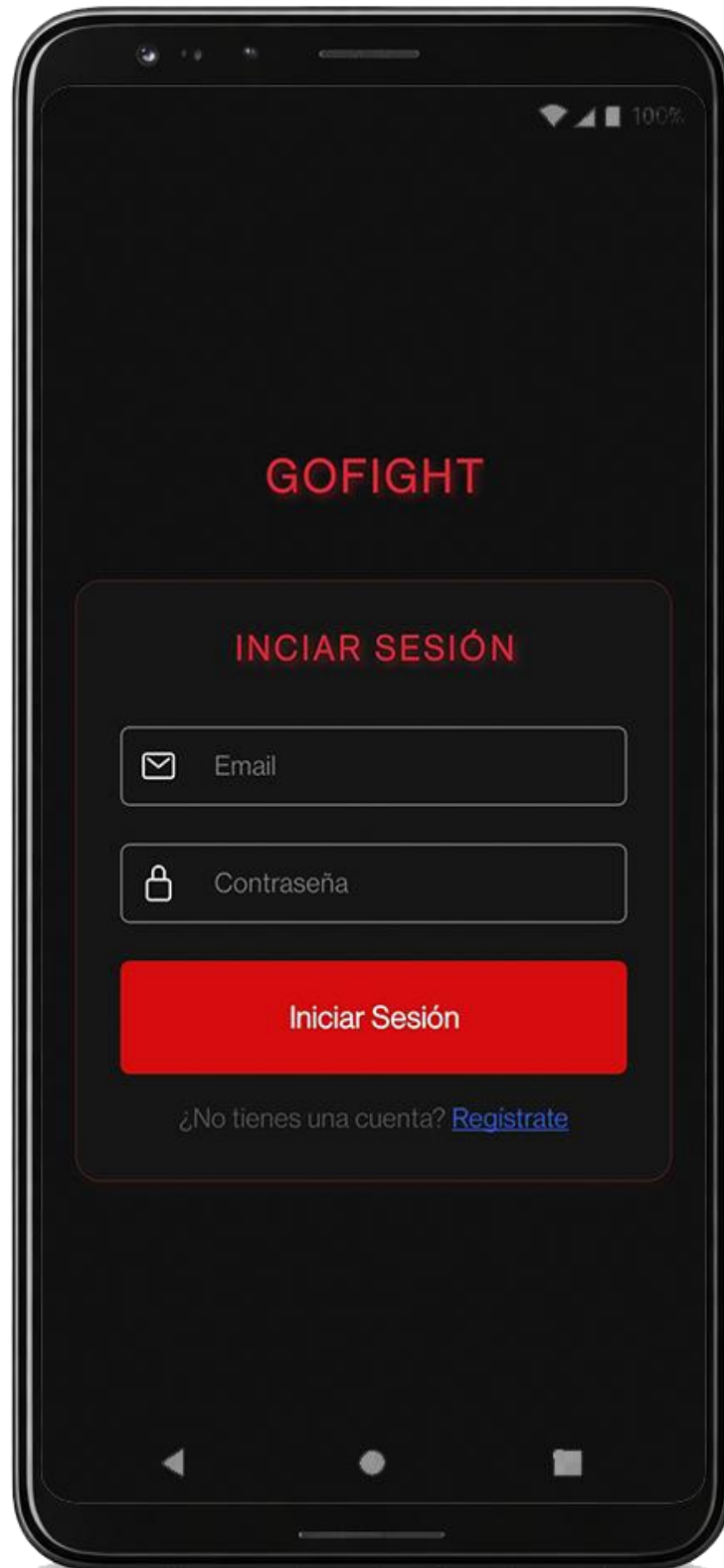


Imagen 6: Captura de pantalla del inicio de sesión.

PANTALLA HOME



Imagen 7: Captura de pantalla del registro del menú de la aplicación.

PANTALLA DE RUTINAS

La pantalla de Rutinas muestra al usuario el listado completo de rutinas de entrenamiento disponibles en la aplicación. Cada rutina se presenta en una tarjeta que incluye el nombre de la rutina, la etiqueta de dificultad (Fácil, Intermedio o Avanzado) y un botón de reproducción para acceder a su contenido. La pantalla incorpora además un buscador en la parte superior que permite filtrar las rutinas por nombre, facilitando la navegación cuando el número de rutinas disponibles es elevado. Al pulsar el botón de reproducir de una rutina, el usuario accede al detalle de los ejercicios que la componen.



Imagen 8: Pantalla de listado de rutinas con buscador y etiquetas de dificultad.

PANTALLA DE EJERCICIOS DE UNA RUTINA

Dentro de cada rutina, la aplicación muestra al usuario la lista de ejercicios que la componen. Cada ejercicio se presenta con una imagen representativa, el nombre del ejercicio, su categoría (por ejemplo, Saco) y un botón para iniciar el temporizador asociado con la duración configurada. Esta pantalla implementa la lógica de secuenciación definida en la base de datos: los ejercicios se muestran según el orden establecido en la tabla intermedia rutinas_ejercicios. Al iniciar cada ejercicio, se activa un contador regresivo que gestiona automáticamente los tiempos de trabajo y descanso, guiando al usuario a través del entrenamiento sin necesidad de interacción adicional.



Imagen 9: Pantalla de ejercicios de una rutina con temporizador de inicio.

PANTALLA DE PROGRESO

La pantalla de Progreso ofrece al usuario una visión global de su actividad dentro de la aplicación. Se muestran cuatro tarjetas de resumen: la racha actual de días consecutivos de entrenamiento, el total de puntos acumulados con acceso directo al ranking, el número de sesiones completadas y el número de rutinas activas. Además, incluye una barra de progreso con el objetivo diario de calorías. Esta pantalla actúa como panel de control del usuario, permitiendo visualizar de un vistazo sus logros y motivando la continuidad del entrenamiento.

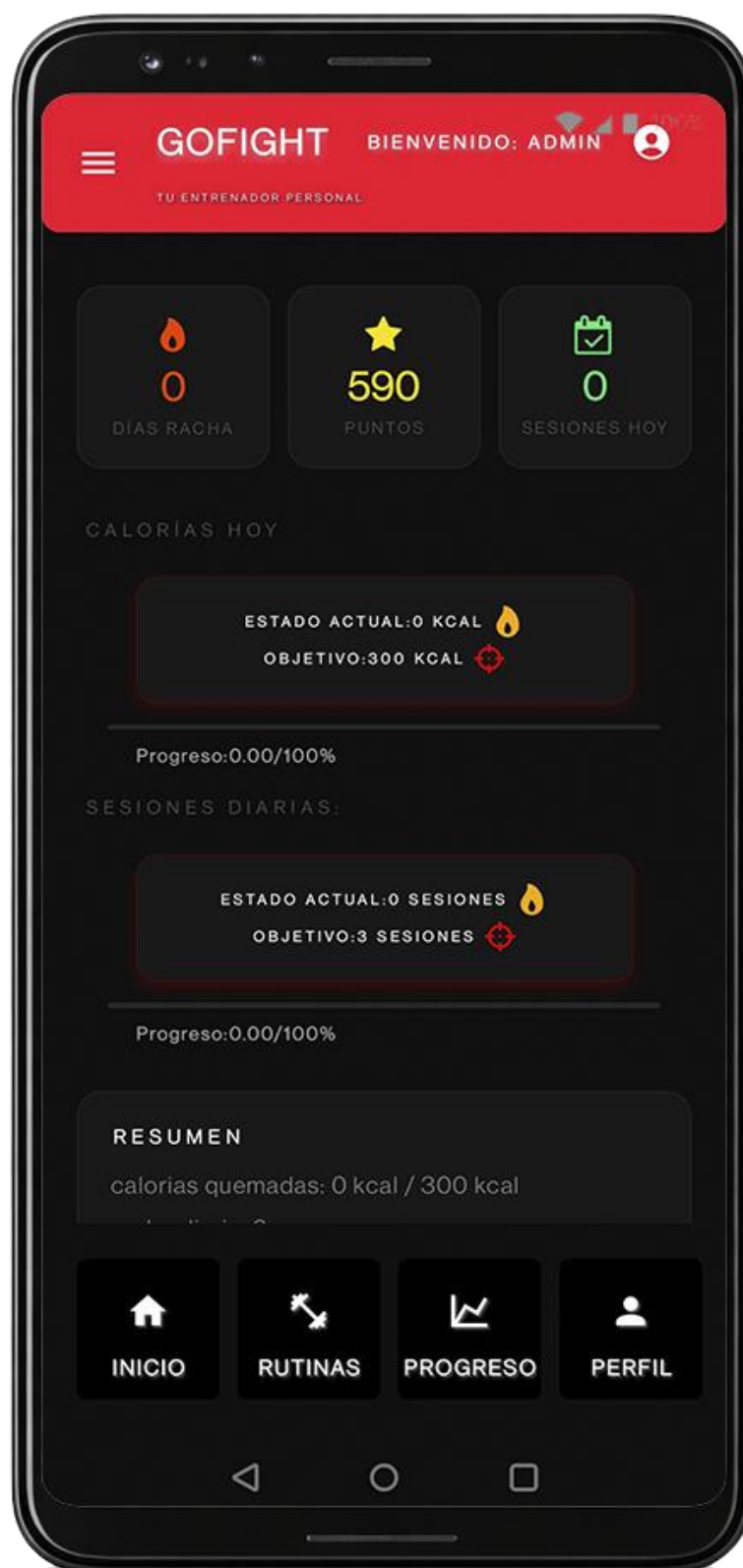


Imagen 10: Pantalla de progreso con resumen de racha, puntos, sesiones y calorías.

PANTALLA DE PERFIL

La pantalla de Perfil permite al usuario consultar y actualizar su información personal. Muestra el avatar junto al nombre de usuario, y una sección con el correo electrónico y la contraseña enmascarada. Un botón permite actualizar los datos. En la parte inferior se presentan las estadísticas del usuario (días de racha y puntos totales) y un botón para cerrar la sesión de forma segura.

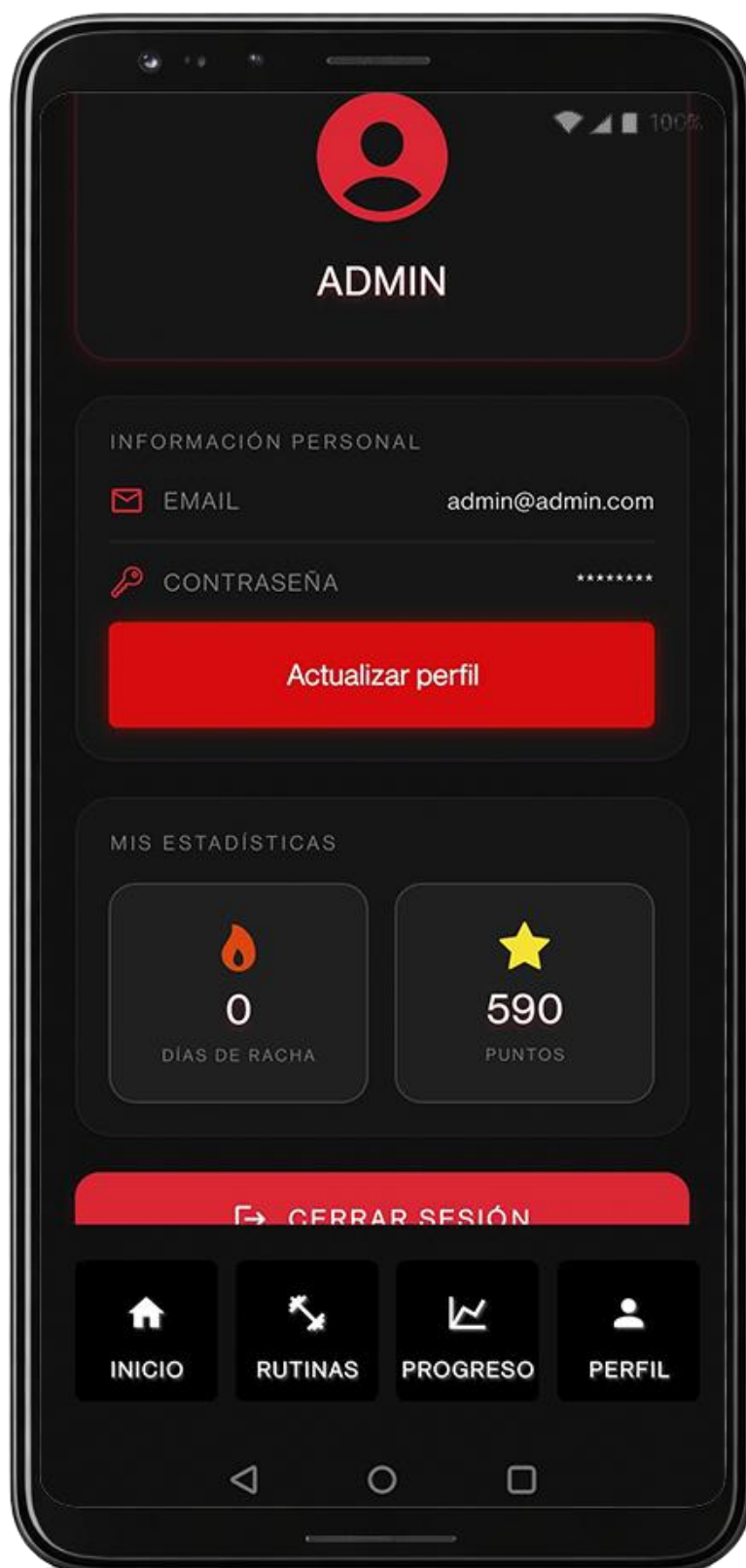


Imagen 11: Pantalla de perfil con información personal, estadísticas y cierre de sesión.

PANTALLA DE RANKING

La pantalla de Ranking muestra un listado ordenado de todos los usuarios según sus puntos acumulados. El primer puesto aparece destacado en dorado, el segundo en plateado y el tercero en bronce. Los puestos siguientes se muestran con fondo oscuro y borde rojo. Cada entrada incluye el nombre del usuario, la cantidad de puntos y el número de posición. La pantalla incorpora un buscador para localizar rápidamente a un jugador específico.

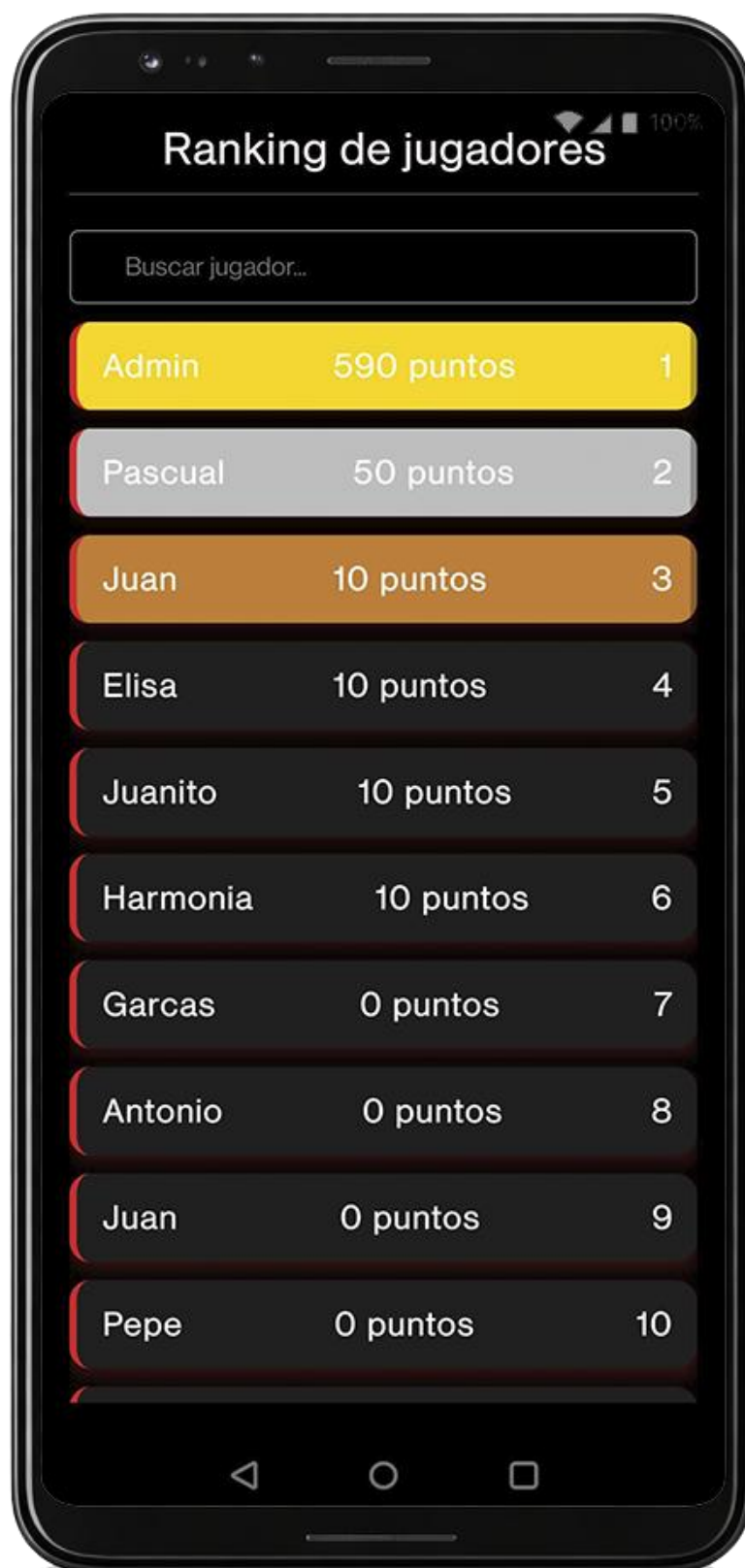


Imagen 12: Pantalla de ranking de jugadores con podio dorado, plateado y bronce.

PANEL DE ADMINISTRACIÓN

La aplicación incorpora un panel de administración exclusivo para usuarios con rol de administrador. Desde esta pantalla, el administrador puede visualizar la lista completa de usuarios registrados en el sistema, mostrando para cada uno su nombre, correo electrónico y rol (admin o user). Para cada usuario se ofrecen tres acciones: eliminar la cuenta, cambiar el rol entre administrador y usuario estándar, y editar la información del perfil. Este panel permite gestionar la base de usuarios de forma centralizada y segura, con acceso restringido mediante autenticación por rol.

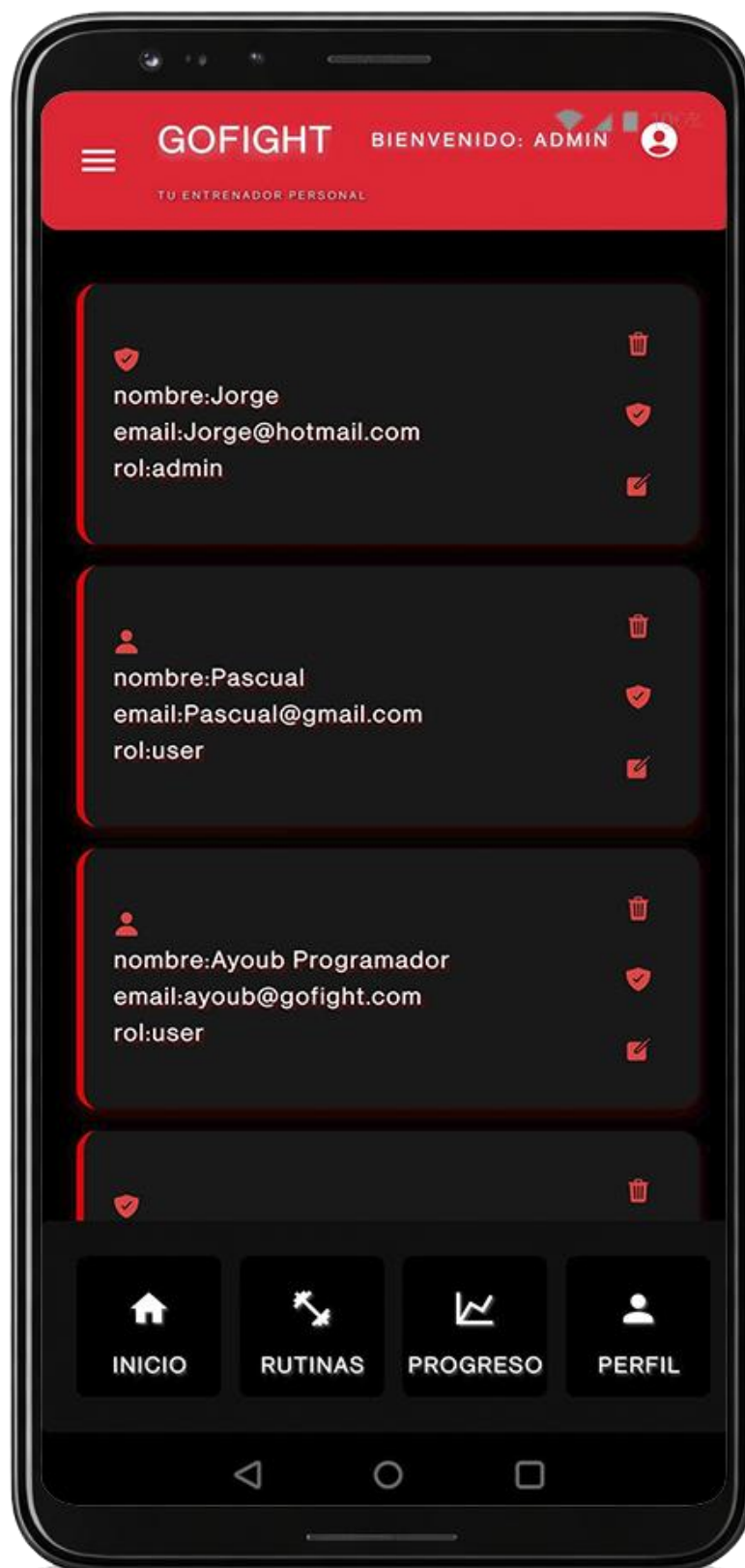


Imagen 13: Panel de administración con gestión de usuarios y roles.

ACTUALIZAR PERFIL

Desde la pantalla de Perfil, al pulsar el botón «Actualizar perfil», se abre un modal superpuesto que permite al usuario modificar sus datos personales. El formulario muestra tres campos: «Nuevo nombre», «Nuevo email» y «Nueva contraseña», cada uno acompañado de su icono representativo. Dos botones de acción permiten guardar los cambios o cancelar la operación volviendo a la pantalla de perfil sin modificar ninguno de los datos almacenados. La validación de los datos introducidos se realiza en el backend antes de persistir cualquier cambio, garantizando la integridad de la información del usuario.



Imagen 14: Modal de actualización de perfil del usuario.

MIS RUTINAS Y CREAR NUEVA RUTINA

La pantalla «Mis Rutinas» muestra al usuario el listado de todas las rutinas que tiene asignadas, presentando para cada una su nombre, la etiqueta de dificultad y un botón rojo de eliminación. En la parte inferior de la pantalla se encuentra el botón «Crear Nueva Rutina», que abre un modal de creación personalizada.

El modal de creación de rutina permite al usuario introducir un nombre para la nueva rutina y seleccionar hasta seis ejercicios de entre todos los disponibles en el sistema. Cada ejercicio se presenta en una tarjeta con su nombre y un botón «+» para añadirlo a la selección. Un contador en tiempo real indica cuántos ejercicios se han seleccionado sobre el máximo permitido (por ejemplo, 0/6). Los botones «Cancelar» y «Crear» permiten descartar o confirmar la nueva rutina respectivamente.

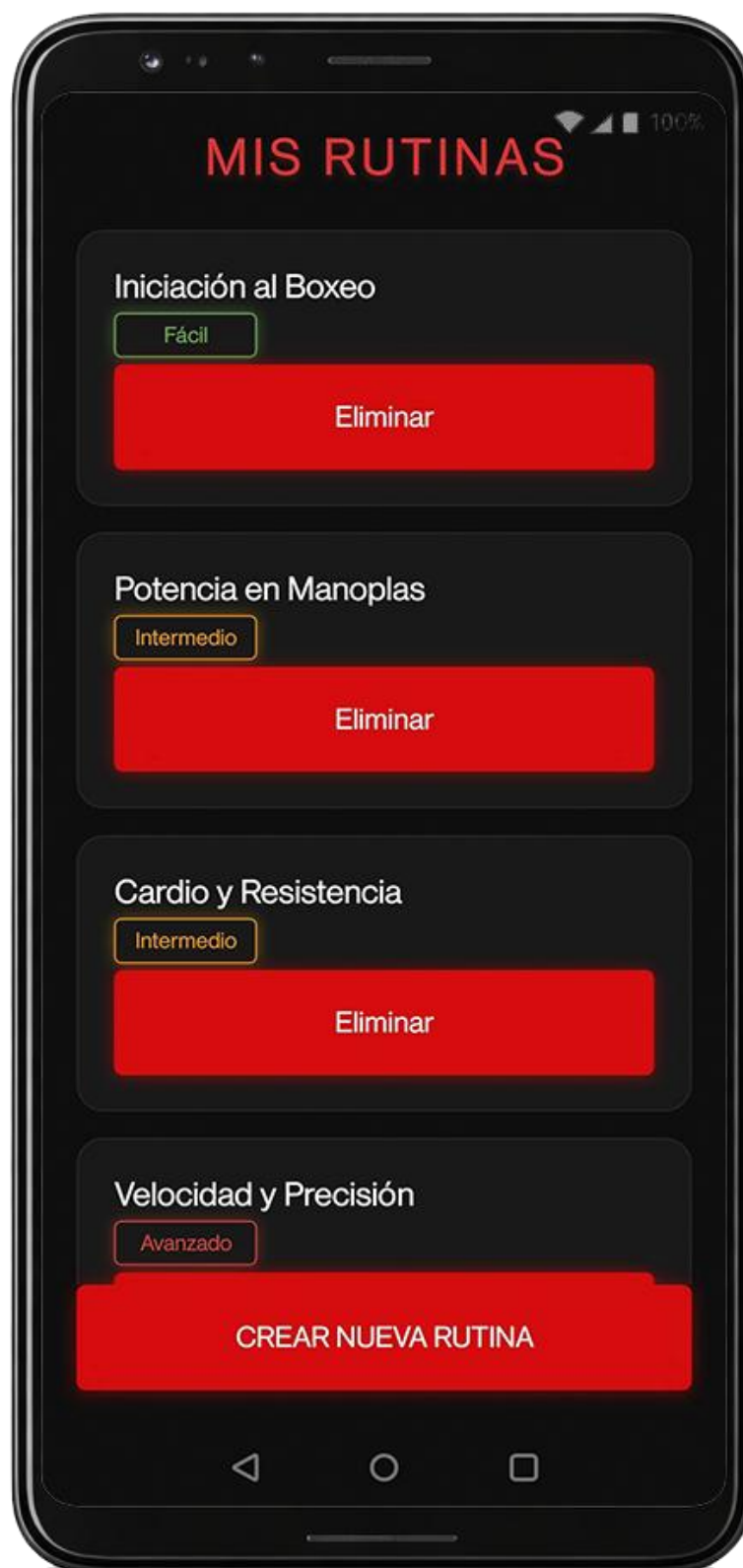


Imagen 15: Pantalla «Mis Rutinas» con opciones de eliminar y crear rutinas.

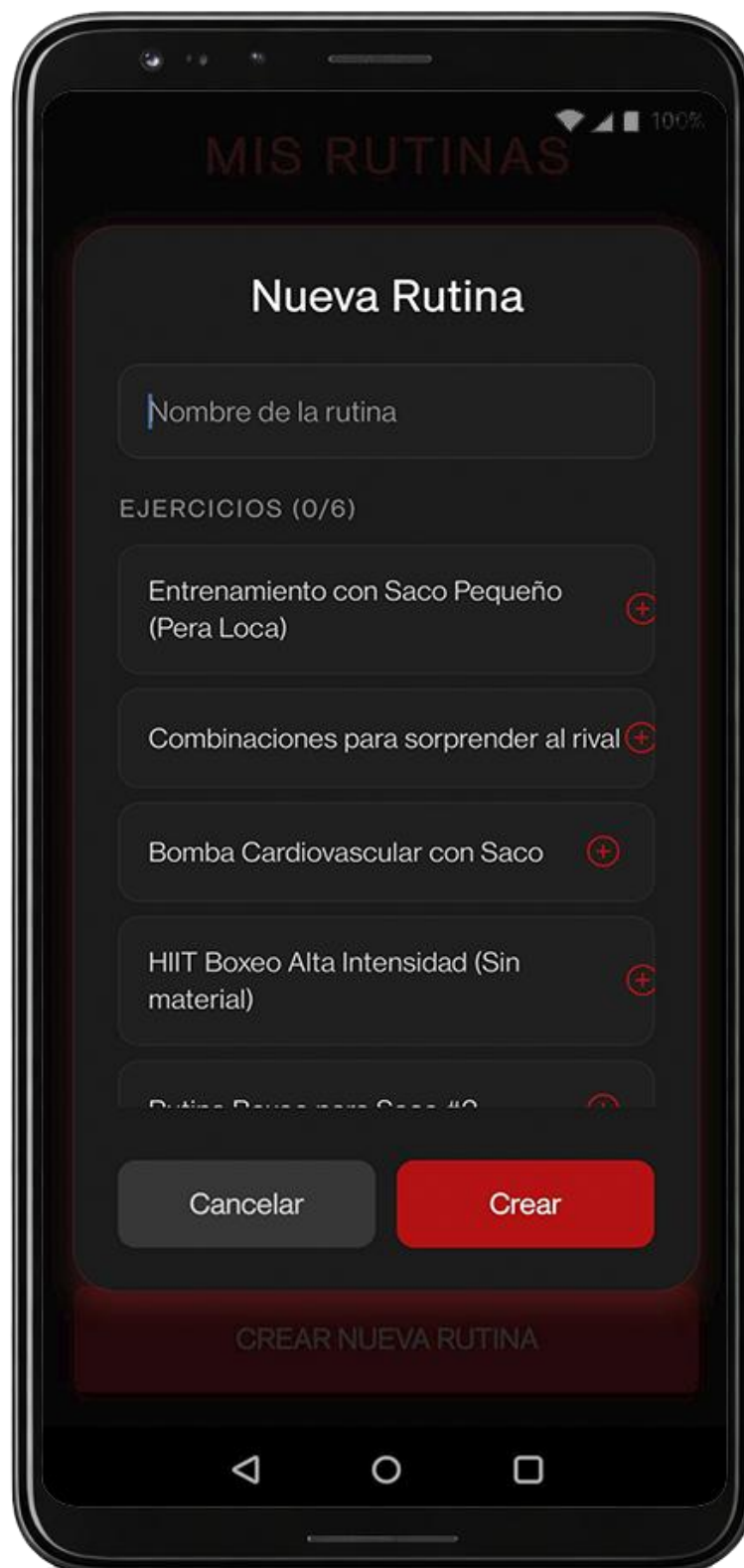
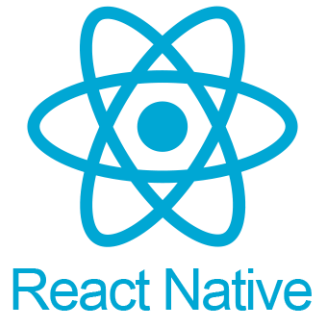


Imagen 16: Modal de creación de nueva rutina personalizada.

TECNOLOGÍA

**React Native.**

React Native es un framework de desarrollo que permite crear aplicaciones móviles utilizando **JavaScript y React**. Una de sus principales ventajas es que permite desarrollar aplicaciones que pueden ejecutarse en diferentes plataformas, como **Android** o incluso adaptaciones para web, utilizando una misma base de código.

En el proyecto GoFight, React Native se ha utilizado para desarrollar la **interfaz de usuario de la aplicación**, incluyendo pantallas como el registro de usuario, el inicio de sesión, la visualización de rutinas de entrenamiento y la consulta de ejercicios.

**Node.js.**

Node.js es un entorno de ejecución para JavaScript que permite desarrollar aplicaciones del lado del servidor. Está diseñado para construir aplicaciones escalables y eficientes, especialmente en sistemas que requieren manejar múltiples conexiones simultáneas.

En este proyecto, Node.js se utiliza para implementar el **backend de la aplicación**, encargándose de procesar las solicitudes de los clientes, aplicar la lógica de negocio y comunicarse con la base de datos.

**Implementación de Seguridad y Lógica de Middlewares.**

Para garantizar la robustez del sistema y la protección de los datos de los usuarios, la API desarrollada con Express.js utiliza una arquitectura basada en **middlewares**. Estas funciones actúan como capas intermedias de control antes de que las peticiones alcancen la lógica de negocio:

- **Control de Acceso mediante JWT:** Se ha implementado un middleware de autenticación que valida el token *JSON Web Token* (JWT) en la cabecera de cada petición. Esto asegura que solo los usuarios autenticados puedan acceder a recursos privados, como su historial de entrenamiento o la edición de su perfil.
- **Validación y Sanitización:** Se han diseñado middlewares específicos para verificar que los datos entrantes cumplen con el esquema definido en Prisma, evitando que lleguen valores nulos o incorrectos a la base de datos, lo que mejora la estabilidad del backend.
- **Gestión de Archivos (Uploads):** El servidor incluye lógica para el procesamiento de archivos multimedia. Mediante el

PROMETEO

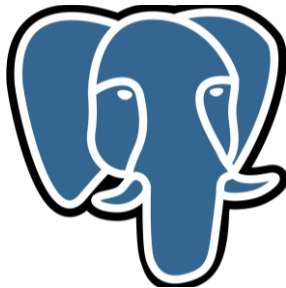
uso de controladores dedicados al almacenamiento, el sistema permite gestionar las imágenes de perfil y los recursos visuales de los ejercicios de forma eficiente.



Express.

Express es un framework para Node.js que facilita la creación de **APIs y servidores web**. Permite definir rutas, gestionar peticiones HTTP y estructurar el backend de forma organizada.

En GoFight se utiliza Express para construir la **API REST** que conecta la aplicación cliente con la base de datos.



PostgreSQL.

PostgreSQL es un sistema de gestión de bases de datos relacional de código abierto que permite almacenar y gestionar información de forma estructurada y segura.

En este proyecto se utiliza PostgreSQL para almacenar los datos relacionados con los **usuarios, rutinas de entrenamiento, ejercicios, historial de sesiones y sistema de gamificación**.



Prisma ORM.

Prisma es una herramienta ORM (Object Relational Mapping) que permite interactuar con bases de datos utilizando código JavaScript o TypeScript en lugar de escribir directamente consultas SQL.

En el proyecto GoFight, Prisma se utiliza para **definir el modelo de datos y gestionar las consultas a la base de datos PostgreSQL**, facilitando el acceso a la información y reduciendo la complejidad del código.

JSON Web Token (JWT).

JSON Web Token es un estándar utilizado para la **autenticación y autorización de usuarios** en aplicaciones web y móviles.



En GoFight se utiliza JWT para gestionar la autenticación de los usuarios, permitiendo que el servidor valide las solicitudes realizadas por los clientes y garantizando que únicamente los usuarios autorizados puedan acceder a determinadas funcionalidades del sistema.

METODOLOGÍA

Metodología usada y justificación de la misma.

Para el desarrollo del proyecto GoFight se ha adoptado una metodología incremental e iterativa, adaptada a las características de un equipo reducido de tres desarrolladores y a la naturaleza del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma (DAM). Esta metodología permite construir el sistema de forma progresiva, añadiendo funcionalidades en cada iteración y validando los resultados antes de continuar con la siguiente fase.

La elección de esta metodología se justifica por varias razones. En primer lugar, al tratarse de un proyecto con requisitos iniciales bien definidos pero con posibilidad de ampliación, el enfoque incremental permite centrar los esfuerzos en entregar un núcleo funcional antes de incorporar características adicionales. En segundo lugar, el trabajo en equipo con tres integrantes facilita la distribución de tareas por módulos: cada desarrollador ha podido trabajar de forma relativamente autónoma en su área, integrándose con el resto del equipo de forma continua mediante el uso de Git como herramienta de control de versiones.

El desarrollo se ha organizado en tres grandes fases o iteraciones, que se detallan a continuación:

Fase 1 – Planificación y diseño (febrero 2025): Durante esta etapa se definieron los requisitos funcionales del sistema, se diseñó la arquitectura cliente-servidor, se elaboraron los diagramas E/R y de clases, y se configuró el entorno de desarrollo. También se inicializó el repositorio Git y se distribuyeron las responsabilidades entre los miembros del equipo: Agustín Linares Carrera se encargó de la coordinación del proyecto y del desarrollo del backend; Ayoub Arramdani participó en el desarrollo de la API REST y la gestión de la base de datos; y Mario Hernández Moreno se centró en el desarrollo del frontend con React Native.

Fase 2 – Desarrollo del núcleo funcional (marzo 2025): Se implementó el módulo de autenticación de usuarios (registro e inicio de sesión con JWT), el módulo de gestión de rutinas y ejercicios, y la integración completa entre el cliente React Native y la API REST. También se desarrolló el sistema de registro de

sesiones de entrenamiento y se realizaron pruebas funcionales de cada endpoint de la API.

Fase 3 – Gamificación, pulido y documentación (abril 2025): Se incorporó el sistema de gamificación (rachas y puntos), el panel de administración de usuarios, el ranking de jugadores y las pantallas de progreso. Se realizaron pruebas de integración, se corrigió la deuda técnica acumulada y se redactó la documentación final del proyecto.

A continuación se presentan dos planificaciones: la estimación inicial realizada al comienzo del proyecto y la planificación real con el tiempo efectivamente dedicado a cada tarea. El tiempo se expresa en horas. Las desviaciones detectadas se deben principalmente a que la implementación del sistema de gamificación y el panel de administración requirieron más tiempo del estimado, mientras que la configuración del entorno resultó más rápida de lo previsto gracias al uso de herramientas modernas como Prisma ORM y Expo para React Native.

Planificación inicial estimada:

Planificación del proyecto (estimación inicial): Análisis y diseño del sistema: 15 horas. Configuración del entorno de desarrollo: 10 horas. Desarrollo del backend (API REST, Node.js, Express): 30 horas. Modelado de base de datos con Prisma y PostgreSQL: 10 horas. Desarrollo del frontend con React Native: 35 horas. Sistema de autenticación con JWT: 8 horas. Sistema de gamificación (puntos y rachas): 10 horas. Pruebas e integración: 10 horas. Documentación del proyecto: 12 horas. Total estimado: 140 horas.

Planificación real (tiempo dedicado):

Planificación del proyecto (tiempo real): Análisis y diseño del sistema: 12 horas. Configuración del entorno de desarrollo: 6 horas. Desarrollo del backend (API REST, Node.js, Express): 32 horas. Modelado de base de datos con Prisma y PostgreSQL: 12 horas. Desarrollo del frontend con React Native: 40 horas. Sistema de autenticación con JWT: 10 horas. Sistema de gamificación (puntos, rachas y ranking): 18 horas. Panel de administración de usuarios: 10 horas. Pruebas e integración: 8 horas. Documentación del proyecto: 15 horas. Total real: 163 horas.

La desviación total ha sido de +23 horas respecto a la estimación inicial. Las principales causas de desviación positiva han sido la incorporación del panel de administración (no contemplado inicialmente), la mayor complejidad del sistema de gamificación y el tiempo adicional dedicado a la documentación. Por otro lado, la configuración del entorno resultó más ágil de lo esperado gracias a la utilización de Expo para React Native y a la automatización de migraciones mediante Prisma.

Diagrama de Gantt.

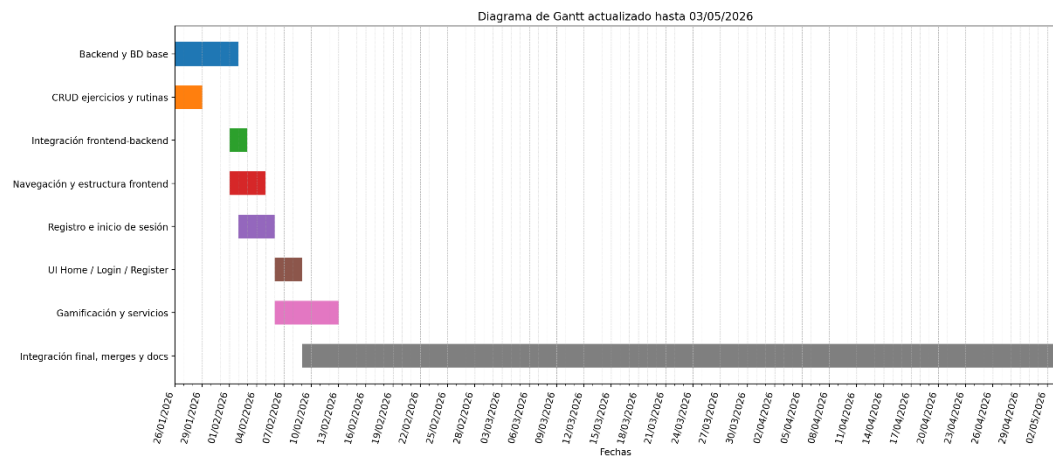
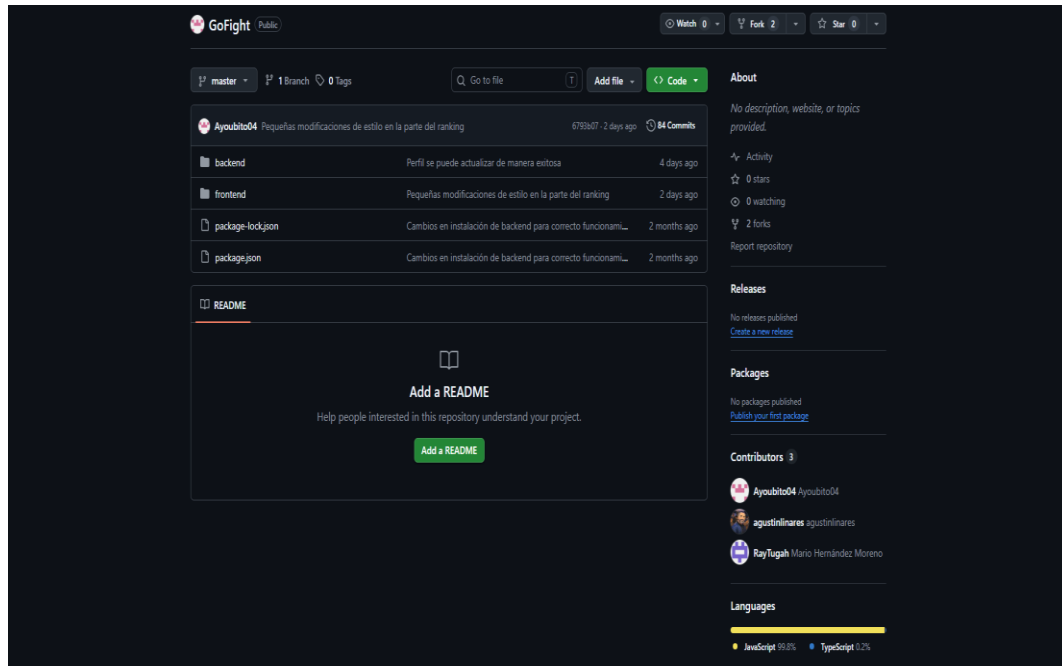


Imagen 16: Diagrama de Gantt sobre la tecnología aplicada en el proyecto durante el tiempo.

README y GIT.



El repositorio del proyecto GoFight se encuentra alojado en GitHub. El repositorio centraliza todo el código fuente del proyecto, incluyendo tanto el backend desarrollado con Node.js y Express como el frontend desarrollado con React Native.

Adicionalmente, existe un repositorio fork del proyecto disponible en <https://github.com/Ayoubito04/GoFight>, que refleja las contribuciones realizadas por los tres miembros del equipo de desarrollo. Los perfiles de GitHub de cada integrante son los siguientes: [@Ayoubito04](#) (Ayoub Arramdani), [@agustinlinares](#) (Agustín Linares Carrera) y [@RayTugah](#) (Mario Hernández Moreno).

El control de versiones se ha gestionado mediante Git, utilizando la rama principal master como rama de producción estable. A lo largo del desarrollo se han realizado commits de forma continua, documentando los cambios más relevantes en cada entrega. Este flujo de trabajo ha permitido mantener un historial claro de la evolución del proyecto y facilitar la coordinación entre los tres integrantes del equipo.

El archivo README.md del repositorio incluye las instrucciones necesarias para la instalación y puesta en marcha del proyecto en un entorno local. En dicho archivo se detallan los requisitos previos del sistema (Node.js, PostgreSQL y npm), el proceso de clonado del repositorio, la configuración de las variables de

entorno mediante el archivo .env, la ejecución de las migraciones de la base de datos con Prisma y el arranque tanto del servidor backend como de la aplicación móvil con Expo.

La imagen anterior muestra una captura del historial de commits del repositorio en GitHub, donde puede apreciarse la frecuencia y distribución de las contribuciones realizadas por los distintos miembros del equipo a lo largo de las tres fases del proyecto. Cada commit refleja el avance incremental del sistema, desde la configuración inicial del entorno hasta la implementación de las funcionalidades más avanzadas como el sistema de gamificación y el panel de administración.

TRABAJO FUTURO

Aunque el proyecto GoFight cumple con los objetivos principales planteados inicialmente, existen diversas mejoras y ampliaciones que podrían implementarse en futuras versiones para ampliar sus funcionalidades y mejorar la experiencia de los usuarios.

En primer lugar, sería interesante incorporar un **sistema de seguimiento del progreso del usuario**, permitiendo registrar datos como peso, repeticiones realizadas o evolución en los ejercicios. De esta forma, el usuario podría visualizar gráficamente su progreso a lo largo del tiempo.

Otra posible mejora sería la incorporación de un **sistema de recomendaciones de rutinas personalizadas**, que sugiera ejercicios o planes de entrenamiento en función del nivel del usuario, sus objetivos o su historial de actividad.

También podría añadirse una **sección social**, donde los usuarios puedan compartir sus rutinas, seguir a otros usuarios o comparar logros, fomentando así la motivación y la interacción dentro de la aplicación.

Desde el punto de vista técnico, el sistema podría evolucionar hacia una **infraestructura en la nube**, permitiendo una mayor escalabilidad y disponibilidad del servicio.

PROMETEO

Por último, se podría desarrollar una **versión web completa de la aplicación**, permitiendo a los usuarios acceder a sus rutinas y progreso también desde un navegador, además de la aplicación móvil.

Además, se contempla como mejora futura la **incorporación de nuevos deportes de contacto dentro de la plataforma**, ampliando el contenido disponible más allá de las disciplinas actuales. Esto permitiría ofrecer rutinas y ejercicios específicos para diferentes modalidades de artes marciales y deportes de combate, aumentando así el alcance y la utilidad de la aplicación para un mayor número de usuarios.

CONCLUSIONES

El desarrollo del proyecto **GoFight** ha permitido diseñar e implementar una aplicación orientada al entrenamiento en deportes de contacto, facilitando a los usuarios el acceso a rutinas y ejercicios de forma organizada y accesible desde dispositivos móviles y web. A lo largo del proyecto se han aplicado los conocimientos adquiridos durante la formación en desarrollo de software, especialmente en el uso de tecnologías modernas para la creación de aplicaciones completas.

Desde el punto de vista técnico, el proyecto ha permitido trabajar con una arquitectura basada en **cliente-servidor**, utilizando **React Native** para el desarrollo del cliente y **Node.js con Express** para el backend, junto con **PostgreSQL** como sistema de gestión de base de datos. Además, el uso de **Prisma ORM** ha facilitado la gestión y modelado de los datos, mientras que la autenticación mediante **JSON Web Token (JWT)** ha permitido implementar un sistema seguro de acceso para los usuarios.

Durante el desarrollo también se ha puesto en práctica el uso de **Git y GitHub** como herramientas de control de versiones, permitiendo gestionar los cambios del proyecto de forma organizada y mantener un historial completo de la evolución del sistema. Esto ha facilitado la planificación del trabajo y la integración de las diferentes funcionalidades desarrolladas.

En términos generales, el proyecto ha cumplido con los objetivos planteados inicialmente, logrando implementar las funcionalidades principales como el registro de usuarios, la autenticación, la visualización de rutinas y la consulta de ejercicios. Además, el sistema se ha diseñado de forma modular, lo que permite ampliar sus funcionalidades en el futuro, como la incorporación de nuevos deportes de contacto o la mejora del seguimiento del progreso de los usuarios.

En conclusión, el desarrollo de **GoFight** ha supuesto una experiencia práctica muy valiosa, permitiendo aplicar conocimientos teóricos en un proyecto real y adquirir experiencia en el desarrollo de aplicaciones completas, desde el diseño inicial hasta la implementación y documentación final del sistema.

PROMETEO

"A día de hoy, el repositorio centraliza una versión funcional del ecosistema GoFight, donde se ha logrado la integración completa entre la base de datos PostgreSQL, el servidor Express y la aplicación móvil, cumpliendo con los hitos de autenticación segura y gestión de tiempos de entrenamiento planteados en los objetivos iniciales."

REFERENCIAS

Facebook Inc. (s. f.). *React Native Documentation*. Recuperado de <https://reactnative.dev/docs/getting-started>

OpenJS Foundation. (s. f.). *Node.js Documentation*. Recuperado de <https://nodejs.org/en/docs>

Express.js. (s. f.). *Express Web Framework Documentation*. Recuperado de <https://expressjs.com/>

PostgreSQL Global Development Group. (s. f.). *PostgreSQL Documentation*. Recuperado de <https://www.postgresql.org/docs/>

Prisma. (s. f.). *Prisma ORM Documentation*. Recuperado de <https://www.prisma.io/docs>

Auth0. (s. f.). *JSON Web Token Introduction*. Recuperado de <https://jwt.io/introduction>

GitHub. (s. f.). *Git Documentation*. Recuperado de <https://docs.github.com/en/get-started>

Chacon, S., & Straub, B. (2014). *Pro Git* (2ª ed.). Apress. Recuperado de <https://git-scm.com/book/en/v2>